ARGUS Cloud Storage Bucket Management

Architecture, Namespace Partitioning & Implementation Specification

> **CORE INVARIANT**
> AI agents, tools, and untrusted principals **never** receive direct read access to raw company files.
> The Cloud Storage layer physically separates immutable raw originals from policy-governed derived views.

1. Storage Topology & Boundary Model

ARGUS implements a multi-tier, tenant-isolated cloud storage model satisfying strict enterprise data governance standards:

Tier / Stage	Storage Path & Namespace	Security, Isolation & Governance Controls
Tier 1: Ingestion & Immutable Originals	`gs://argus-tenant-{org}/originals/{sha256}.{ext}`	• Ingestion from Google Drive, SharePoint, Web Upload, Datastores. • Addressed exclusively by SHA-256 hash (content-addressable). • WORM Lock (Object Retention Hold) prevents tampering & deletion. • CMEK encryption with restricted, customer-managed KMS key.
Authoritative Transformation	*In-Memory Compute Pipeline (RedactCore Service)*	• Isolated Cloud Run compute container. • Reads immutable original in read-only stream. • AST Parsing + Policy-Driven In-Memory True Redaction. • Independent Leak Verification Gate (Zero-Leakage Guarantee).
Tier 2: Role-Guarded Safe Projections	`gs://argus-tenant-{org}/views/{role_id}/{view_id}.{ext}`	• Physically distinct derived files synthesized per role / purpose. • E.g., Sales View (PII purged), Finance View (contacts masked). • Only verified sanitized views are written to this path.
Tier 3: Cryptographic Manifests	`gs://argus-tenant-{org}/manifests/{view_id}.json`	• Cryptographic audit ledger with finding digests & policy seeds. • Never contains raw PII or secret entity values. • Verifiable proof of compliance for enterprise compliance auditors.
Delivery & Gateway Layer	*Cerberus Gateway & ArgusBC Control Plane*	• Short-lived V4 Signed URLs (30 min for humans, 10 min for AI agents). • Zero-intermediary streaming: client reads directly from GCS. • Workspace vector index RAG embeddings pull solely from Tier 2 views.

Storage Namespace Layout

All objects are partitioned within the workspace bucket boundary (`gs://argus-tenant-<org_id>/`):

```
gs://argus-tenant-org-hukurou/
|-- originals/                              # TIER 1: Immutable Originals (WORM Enclave)
|   |-- 4a8e2b...f1c8.pdf                   # Addressable solely by content SHA-256
|   \-- 9b12d4...a3e2.docx                  # WORM locked (no overwrite, no delete)
|
|-- views/                                  # TIER 2: Role-Guarded Safe Projections
|   |-- role_sales/
|   |   \-- view_8f3a910c.pdf               # PII & compensation purged
|   |-- role_finance/
|   |   \-- view_2b819e4a.pdf               # Compensation visible; contacts masked
|   \-- role_reviewer/
|       \-- view_7d12f45e.pdf               # Flagged figures marked for audit
|
|-- manifests/                              # TIER 3: Cryptographic Audit Trail
|   |-- view_8f3a910c.json                  # Finding digests, decision seeds, zero PII
|   \-- view_2b819e4a.json
|
\-- staging/                                # Ephemeral Ingestion Chunking
    \-- upload_tmp_...                      # Auto-expire after 24h (Lifecycle Rule)
```

2. Core Actions & Control Flows

Flow A: Ingestion & Immutable WORM Placement

How raw documents enter the enclave without risking tampering, collision, or silent mutation:

Step	Actors & Path	Action Description & Verification Logic
1	**Admin / Ingestion [to] ArgusBC**	Admin consents to document ingestion via Google Drive sync or direct web upload.
2	**ArgusBC Internal**	Streams raw byte stream and calculates authoritative SHA-256 content checksum.
3	**ArgusBC [to] GCS originals/**	Checks existence of `originals/{sha256}.{ext}`. • *If exists:* Confirms content collision and deduplicates without redundant write. • *If new:* Executes HTTP PUT to `originals/{sha256}.{ext}` with source metadata, then applies an **Object Retention Hold (WORM lock)**.
4	**ArgusBC [to] Firestore**	Records `WorkspaceImmutableFile` entry with hash, byte size, source ref, and retention hold metadata.
5	**ArgusBC [to] Admin / User**	Returns confirmation: document safely ingested, content-addressed, and permanently locked against modification.

Flow B: Policy-Driven View Synthesis & Storage

How RedactCore transforms copies in isolated memory and persists verifiable derivatives:

Step	Actors & Path	Action Description & Verification Logic
1	**Requester / Agent [to] Cerberus**	Requests document representation specifying caller principal, role (e.g. Sales), and declared purpose.
2	**Cerberus [to] RedactCore Engine**	Triggers `POST /api/views` passing `source_sha256`, `policy_id`, and subject credentials.
3	**RedactCore [to] GCS originals/**	Fetches immutable original byte stream in **strict read-only** mode.
4	**RedactCore Internal**	1. Parses AST and detects sensitive entity locations. 2. Applies True Redaction to in-memory byte copy (purging PII, tokens, or compensation data). 3. Runs independent **Leak Verification** on final transformed bytes.
5	**RedactCore [to] GCS & Cerberus**	• *If leak check fails:* Aborts write, emits alert, returns 500 View Suppressed (Zero-Leak Guarantee). • *If leak check passes:* Writes sanitized file to `views/{role}/{view_id}.{ext}` and audit digest to `manifests/{view_id}.json`. Returns ViewManifest & object key to Cerberus.

Flow C: Presigned URL Retrieval (Zero-Intermediary Streaming)

Preventing multi-megabyte binary payloads from clogging API gateway memory:

Step	Actors & Path	Action Description & Verification Logic
1	**Agent / Human [to] ArgusBC Gateway**	Invokes `GET /api/views/{view_id}/download-url`.
2	**ArgusBC Gateway Internal**	Verifies caller authorization, tenant boundary, and active session permissions for `{view_id}`.
3	**ArgusBC Gateway [to] GCS API**	Issues Google Cloud Storage V4 Signed URL (valid for 30 minutes for humans / 10 minutes for agents, restricted to HTTP GET).
4	**ArgusBC Gateway [to] Caller**	Returns Signed URL payload (`https://storage.googleapis.com/...`).
5	**Caller [to] Google Cloud Storage**	Direct HTTPS GET request to GCS; receives streamed safe derived bytes directly without intermediary proxy overhead.

3. Step-by-Step Implementation Plan

Phase 1: Real Cloud Storage Provider for RedactCore

- Create `redact_core/storage_gcs.py` implementing `GCSDocumentStore` conforming to the existing `DocumentStore` protocol.
- Support hybrid local/cloud modes via `REDACTCORE_STORAGE_BACKEND=gcs` (falling back to local filesystem in offline unit tests).
- Add Google Cloud Storage V4 signed URL generator for direct, zero-proxy streaming.

Phase 2: Bucket Lifecycle & Security Policies in Terraform / GCP

- **WORM Lock:** Configure default object retention hold or bucket retention policy on `originals/`.
- **Lifecycle Rules:** Configure auto-deletion for `staging/` scratch uploads after 24 hours.
- **Uniform Bucket-Level Access:** Disable legacy ACLs; enforce IAM-only access control.
- **KMS Encryption:** Configure Customer-Managed Encryption Keys (CMEK) for tenant-isolated data.

Phase 3: ArgusBC Bucket Management & Live UI Integration

- Connect `ArgusBC/app/documents/page.tsx` to live bucket state in Google Cloud project `argus-guardian`.
- Display real storage usage metrics, object retention hold status, and KMS encryption metadata.
- Wire the **"Role-Guarded Buckets"** tab so users can inspect real transformed files partitioned per role.

4. Architectural Decisions (Approved)

1. **Bucket Granularity:**
   - **Decision: Option A (Single Tenant Bucket with Prefix Namespaces)** (`gs://argus-tenant-<org_id>/originals/`, `.../views/`, `.../manifests/`, `.../staging/`). Unified lifecycle policies, lower operational overhead, and streamlined IAM bindings.
2. **Signed URL Expiration (TTL):**
   - **30 minutes** for human inspection, **10 minutes** for AI agent context ingestion.
3. **WORM Retention Period:**
   - **365 days** (Enterprise compliance retention hold on `originals/`).